

HLD Chapter 3: Databases, Storage Engines, Sharding & Consistent Hashing

Core Learning Objective: Master database architecture at planetary scale. Understand B-Trees vs LSM-Trees, replication topologies, horizontal partitioning (sharding), and the mathematics of Consistent Hashing with virtual nodes.

1. Storage Engines Under the Hood: B-Trees vs LSM-Trees

Every database's performance characteristics are dictated by its underlying on-disk storage engine:

Metric	B-Tree (PostgreSQL, MySQL InnoDB)	LSM-Tree (Cassandra, RocksDB, ScyllaDB)
Primary Optimization	Read-Heavy workloads	Write-Heavy workloads
Write Mechanism	In-place random page updates	Append-only sequential writes to MemTable + WAL
Storage Structure	Self-balancing tree of fixed-size disk blocks ($4\text{KB} - 16\text{KB}$)	Memory buffer (MemTable) flushed to sorted disk files (SSTables)
Write Amplification	Higher (full page rewrite on single row update)	Lower for writes; compaction overhead later
Lookup Latency	$O(\log N)$ predictable disk seeks	In-memory MemTable check $\rightarrow$ Bloom Filter check $\rightarrow$ SSTable seek

```

flowchart TD
    subgraph LSM_Tree_Write_Path ["LSM-Tree Write Path (Zero Random Disk Seeks)"]
        Write["Write Request (k, v)"] --> WAL["1. Append to Write-Ahead Log (WAL on Disk)"]
        WAL --> MemTable["2. Insert into In-Memory MemTable (SkipList / Red-Black Tree)"]
        MemTable -->|When full (e.g. 64 MB)| Flush["3. Sequential Flush to Disk"]
        Flush --> SSTable["Immutable SSTable (Sorted String Table)"]
        SSTable --> Compaction["Background Compaction (Merges duplicate keys)"]
    end
    end

```

2. Replication Models: Leader vs Leaderless

```

flowchart LR
    subgraph Single_Leader ["Single-Leader (Master-Replica)"]
        ClientW["Client Write"] --> Leader["Leader Node (Accepts Writes)"]
        Leader -->|Sync/Async Replication| Follower1["Follower (Read Only)"]
        Leader -->|Sync/Async Replication| Follower2["Follower (Read Only)"]
    end
    end

    subgraph Leaderless ["Leaderless (Amazon Dynamo / Cassandra)"]
        Client["Client Coordinator"] --> N1["Node 1"]
        Client --> N2["Node 2"]
        Client --> N3["Node 3"]
        Note["Quorum Formula: R + W > N<br/>Guarantees overlap on latest written replica"]
    end
    end

```

The Quorum Mathematics ($R + W > N$)

In leaderless systems:

- N = Number of replicas stored for each key.
- W = Number of write acknowledgements required before reporting success.
- R = Number of replicas queried on read.
- **If $R + W > N$:** By the Pigeonhole Principle, at least **one** of the queried nodes in R is guaranteed to contain the most recent write.

3. Sharding & Horizontal Partitioning

When data exceeds the disk capacity or throughput limits of a single machine ($> 2 \text{ TB}$ or $> 20,000 \text{ QPS}$), the database must be partitioned:

Partitioning Strategies:

1. **Range-Based Partitioning:** (e.g. Users A-C on Node 1, D-F on Node 2).
 - *Flaw:* Causes massive **hot spots** (e.g. Celebrity accounts or localized surges).
2. **Hash-Based Partitioning:** $\text{Node} = \text{hash}(\text{key}) \bmod N$.
 - *Fatal Flaw:* When a node is added or removed ($N \rightarrow N + 1$), nearly 100% of keys hash to a new node, causing a catastrophic full-cluster re-indexing storm!

4. Consistent Hashing with Virtual Nodes (The Standard)

Consistent Hashing maps both **nodes** and **data keys** to an abstract circular ring ($[0, 2^{32}-1]$).

```

flowchart TD
    subgraph Ring ["Consistent Hash Ring (0 to 2^32 - 1)"]
        NodeA["Node A (Token 100)"]
        Key1["Key 'user_88' (Token 150)"]
        NodeB["Node B (Token 300)"]
        Key2["Key 'user_99' (Token 450)"]
        NodeC["Node C (Token 600)"]

        Key1 -.->|Walks clockwise to nearest node| NodeB
        Key2 -.->|Walks clockwise to nearest node| NodeC
    end
  
```

Why It Eliminates Re-indexing Storms

When a new node is inserted into the ring, **only the keys immediately preceding the new node are moved**; all other nodes remain completely untouched! On average, only $\frac{K}{N}$ keys are relocated.

Eliminating Hotspots: Virtual Nodes (vnodes)

If physical nodes are placed randomly on the ring, load distribution is unequal. **Solution:** Each physical server is assigned multiple **Virtual Nodes** (e.g., 256 tokens scattered across the ring: Node_A#1, Node_A#2, etc.). This ensures statistical uniformity across all physical hardware.

Complete Consistent Hash Ring Implementation in Python

```
import hashlib
import bisect
from typing import List, Optional

class ConsistentHashRing:
    def __init__(self, replicas: int = 100):
        self.replicas = replicas # Number of virtual nodes per physical server
        self.ring: List[int] = [] # Sorted token list
        self.vnode_to_node: dict[int, str] = {} # Token → Physical Node mapping

    def _hash(self, key: str) → int:
        return int(hashlib.md5(key.encode('utf-8')).hexdigest(), 16)

    def add_node(self, node: str) → None:
        for i in range(self.replicas):
            vnode_key = f"{node}#vnode_{i}"
            token = self._hash(vnode_key)
            bisect.insort(self.ring, token)
            self.vnode_to_node[token] = node
        print(f"[Hash Ring] Added physical node '{node}' ({self.replicas} vnodes).")

    def remove_node(self, node: str) → None:
        for i in range(self.replicas):
            vnode_key = f"{node}#vnode_{i}"
            token = self._hash(vnode_key)
            idx = bisect.bisect_left(self.ring, token)
            if idx < len(self.ring) and self.ring[idx] == token:
                del self.ring[idx]
                del self.vnode_to_node[token]
        print(f"[Hash Ring] Removed physical node '{node}'.")

    def get_node(self, key: str) → Optional[str]:
        if not self.ring:
            return None
        token = self._hash(key)
        # Binary search for the first node clockwise ≥ token
        idx = bisect.bisect_right(self.ring, token)
        if idx == len(self.ring):
            idx = 0 # Wrap around the circle
        return self.vnode_to_node[self.ring[idx]]
```

```
# Verification
ring = ConsistentHashRing(replicas=150)
ring.add_node("DB_Shard_A")
ring.add_node("DB_Shard_B")
ring.add_node("DB_Shard_C")

print("Key 'user_10291' maps to:", ring.get_node("user_10291"))
print("Key 'order_99812' maps to:", ring.get_node("order_99812"))
```